

What Do They Actually Need?

Requirements gathering begins

Kurosawa Digital, open-plan floor. Ren and Hina sit across from three end users — a UX designer, a sales manager, and a warehouse supervisor — who will actually use the XR platform. Each one has a different idea of what "the platform" means.

Ren (蓮) "Thanks for making time. We're here to understand what you actually need this thing to do. Not what the spec says — what *you* need."

→ *Requirements gathering* starts by talking to the people who will use the product, not just the people who approved the budget. User needs often differ from documented **requirements**.

Hina (陽菜) "Um, I'm taking notes on everything — is it okay if I record the session too? I don't want to miss anything important!"

Ren (蓮) *(to Hina, after the UX designer asks for real-time collaboration and the sales manager asks for offline mode)* "Write that down — they just gave us two **requirements** that directly contradict each other."

→ Conflicting requirements are normal during gathering. The PM's job isn't to resolve them on the spot — it's to capture them accurately and flag the conflicts for later prioritization.

Hina (陽菜) "The warehouse supervisor wants something completely different — he just needs a training mode with step-by-step overlays. That doesn't sound like the same product at all."

→ Different **stakeholders** have different needs. A good requirements process surfaces these gaps early, before they become design conflicts.

Ren (蓮) "It's all the same platform — it just means the **scope statement** needs to account for multiple user types. Each **work package** in the **WBS** might serve a different audience."

→ Requirements inform scope. When user needs are diverse, the *WBS* must be structured to handle multiple use cases without inflating the **scope baseline** beyond what's approved.

Hina (陽菜) "Is it okay if I ask — how do we know which requirements are the most important? There are so many!"

Ren (蓮) "That's the right question. We rank them — must-have versus nice-to-have. And we check each one against the **stakeholder register** to see who's asking and how much **influence** they carry."

→ Requirements prioritization uses the stakeholder analysis from Unit 5. A requirement from a high-**influence stakeholder** with a high **engagement level** carries more weight in scope decisions.

Hina (陽菜) "So the **power/interest grid** isn't just for knowing who to talk to — it's for knowing whose requirements matter most?"

Ren (蓮) "Now you're getting it. Okay, let's organize this mess. Group everything by user type, flag the conflicts, and—"

Hina (陽菜) "Oh! Ren, Sakurai-san just sent you an email. She says it's a... 'requirements traceability template, for reference.'"

*Ren opens the attachment. It's thorough — columns for source, priority, status, linked **deliverables**. She must have spent time on this. He starts typing a reply, deletes it, types again.*

Concept Discovery — Requirements Gathering

1 The Mess Is the Method. When Ren sat down with three users and got three different answers, that wasn't a failure — that's requirements gathering working correctly. The goal is to surface *all* needs, including contradictions, before design begins. Techniques include interviews (like this session), workshops, questionnaires, observation, and prototype feedback. The key discipline: capture first, evaluate later. Ren's instinct to "organize the mess" only after hearing everyone is exactly right.

Tip Never resolve conflicts during the gathering session. Record every requirement with its source. Prioritization happens afterward, informed by the **stakeholder register** and project constraints.

2 Requirements Connect Everything. Hina's insight — that the **power/interest grid** helps prioritize requirements — shows how PM concepts build on each other. Requirements flow from **stakeholders** (Unit 3), get bounded by the **scope statement** (Unit 6), and decompose into **work packages** through the **WBS** (Unit 6). Saki's traceability template adds the crucial link: every requirement traced to its source stakeholder, its priority, and its corresponding **deliverable**. When requirements are traceable, nothing gets lost — and nothing sneaks in.

Tip A requirements traceability matrix links each requirement to its origin, its status, and the deliverable that satisfies it. Build it early; update it always.